Algoritmos de optimización - Seminario

Nombres y Apellidos:

- Carlos Eduardo Chavarria Mujica
- Ivan Luis Eduardo Rojas Rojas

Url: https://github.com/08edu02/03MIAR_04_A_2026_Proyecto_TrabajoPractico
Problema:

> 2. Organizar los horarios de partidos de La Liga

Descripción del problema: Desde la La Liga de fútbol profesional se pretende organizar los horarios de los partidos de liga de cada jornada. Se conocen algunos datos que nos deben llevar a diseñar un algoritmo que realice la asignación de los partidos a los horarios de forma que maximice la audiencia. Se deben respetar las restricciones de horarios disponibles, coeficientes de reducción por horario, penalizaciones por coincidencias y la obligatoriedad de asignar al menos un partido el viernes y otro el lunes.

(*) La respuesta es obligatoria

Organizar los horarios de partidos de La Liga

Desde la La Liga de fútbol profesional se pretende organizar los horarios de los partidos de liga de cada jornada. Se conocen algunos datos que nos deben llevar a diseñar un algoritmo que realice la asignación de los partidos a los horarios de forma que maximice la audiencia.

Horarios disponibles

Los horarios disponibles se conocen a priori y son los siguientes:

Día	Horarios
Viernes	20:00
Sábado	12:00, 16:00, 18:00, 20:00
Domingo	12:00, 16:00, 18:00, 20:00
Lunes	20:00

Clasificación de equipos

En primer lugar se clasifican los equipos en tres categorías según el número de seguidores (que tiene relación directa con la audiencia). Hay:

- **3 equipos** en la categoría A
- **11 equipos** en la categoría B
- **6 equipos** en la categoría C

Audiencia base por categorías

Se conoce estadísticamente la audiencia que genera cada partido según los equipos que se enfrentan y en horario de sábado a las 20h (el mejor en todos los casos):

Categoría	A	B	C
A	2 Millones	1.3 Millones	1 Millón
B	---	0.9 Millones	0.75 Millones

Coeficientes de reducción por horario

Si el horario del partido no se realiza a las 20 horas del sábado se sabe que se reduce según los coeficientes de la siguiente tabla:

Horario	Viernes	Sábado	Domingo	Lunes
12:00	---	0.55	0.45	---
16:00	---	0.70	0.75	---
18:00	---	0.80	0.85	---
20:00	0.40	1.00	1.00	0.40

Restricciones obligatorias

Debemos asignar obligatoriamente siempre un partido el viernes y un partido el lunes.

Penalización por coincidencia de horarios

Es posible la coincidencia de horarios pero en este caso la audiencia de cada partido se verá afectada y se estima que se reduce en porcentaje según la siguiente tabla dependiendo del número de coincidencias:

Coincidencias	-%
0	0%
1	25%
2	45%
3	60%
4	70%
5	75%
6	78%
7	80%
8	80%

Ejemplo de cálculo de una jornada

Los cálculos asociados a una jornada de ejemplo se realizan según se muestra en la siguiente tabla:

Partido	Categorías	Horario	Base (Mill)	Ponderación	Base × Ponderación	Corrección Coincidencia
Celta vs Real Madrid	B–A	V 20:00	1.30	0.40	0.52	0.52
Valencia vs R. Sociedad	B–A	S 12:00	1.30	0.55	0.72	0.72
Mallorca vs Eibar	C–C	S 16:00	0.47	0.70	0.33	0.33
Athletic vs Barcelona	B–A	S 18:00	1.30	0.80	1.04	1.04
Leganés vs Osasuna	C–C	S 20:00	0.47	1.00	0.47	0.47
Villarreal vs Granada	B–C	D 16:00	0.75	0.75	0.56	0.42
Alavés vs Levante	B–B	D 16:00	0.90	0.75	0.68	0.51
Espanyol vs Sevilla	B–B	D 18:00	0.90	0.85	0.77	0.77
Betis vs Valladolid	B–C	D 20:00	0.75	1.00	0.75	0.75
Atlético vs Getafe	B–B	L 20:00	0.90	0.40	0.36	0.36

Audiencia Total de la Jornada: 5.88 millones

(*)¿Cuantas posibilidades hay sin tener en cuenta las restricciones?

¿Cuantas posibilidades hay teniendo en cuenta todas las restricciones.

Respuesta

Nota: El enunciado indica "asignar obligatoriamente siempre un partido el viernes y un partido el lunes". Se interpreta como "al menos uno". Si la restricción fuera "exactamente uno", el cálculo sería distinto ($10 \times 9 \times 8^8$).

Sin restricciones: Cada uno de los 10 partidos puede asignarse a cualquiera de los 10 horarios disponibles.

$$10^{10} = 10.000.000.000$$

Son 10.000 millones de posibilidades.

Con restricciones (Al menos uno el viernes y al menos uno el lunes): Utilizamos el principio de inclusión-exclusión:

- Total sin restricciones: 10^{10}
- Sin partidos el viernes: 9^{10}
- Sin partidos el lunes: 9^{10}
- Intersección (ni viernes ni lunes): 8^{10}

Fórmula: $10^{10} - 9^{10} - 9^{10} + 8^{10}$

In [1]:
```
total_sin_restricciones = 10**10
sin_viernes = 9**10
sin_lunes = 9**10
ni_viernes_ni_lunes = 8**10

total_con_restricciones = total_sin_restricciones - sin_viernes - sin_lunes + ni_viernes_ni_lunes
print(f"Posibilidades con restricciones: {total_con_restricciones}")
# Resultado: 4100173022
```

```
Posibilidades con restricciones: 4100173022
```

Modelo para el espacio de soluciones

(*) ¿Cual es la estructura de datos que mejor se adapta al problema? Argumentalo.(Es posible que hayas elegido una al principio y veas la necesidad de cambiar, argumentalo)

Respuesta

Inicialmente al analizar el problema, tratamos de hacer un planteamiento práctico sin cosiderar tanto la eficiencia, que nos podría tardar un poco mas en diseñarla.

Una opción fue realizar una matriz de 10x10 que relacione horarios y partidos, pero el problema no es de asignación uno a uno, y además el número de horarios es fijo. Otra opción era usar solo listas y hacer count() para saber cuántos coinciden, pero eso era muy ineficiente porque cada vez que se use, tendría que recontarlo otra vez.

Entonces, replanteando, buscamos la forma de usar también diccionarios. Como tenemos 10 partidos, podemos enviarlos a una lista y cada partido podría representarse en un diccionario con sus datos; tendríamos otros diccionarios para asignar los horarios, coeficientes de ponderación y valores de penalización. Era lo más sencillo y rápido de hacer, porque el problema está planteado con tablas y es menos complejo visualizarlo así.

En ese sentido tendríamos la siguiente estructura:

In [2]:
```
partidos = [
    {
        'id': 0,
        'local': 'Celta',
        'visitante': 'Real Madrid',
        'categoria_local': 'B',
        'categoria_visitante': 'A',
        'base': 1.3  # audiencia base en millones de espectadores
    },
    {
        'id': 1,
        'local': 'Valencia',
        'visitante': 'R. Sociedad',
        'categoria_local': 'B',
        'categoria_visitante': 'A',
        'base': 1.3
    },
    {
```

```
        'id': 2,
        'local': 'Mallorca',
        'visitante': 'Eibar',
        'categoria_local': 'C',
        'categoria_visitante': 'C',
        'base': 0.47
    },
    {
        'id': 3,
        'local': 'Athletic',
        'visitante': 'Barcelona',
        'categoria_local': 'B',
        'categoria_visitante': 'A',
        'base': 1.3
    },
    {
        'id': 4,
        'local': 'Leganés',
        'visitante': 'Osasuna',
        'categoria_local': 'C',
        'categoria_visitante': 'C',
        'base': 0.47
    },
    {
        'id': 5,
        'local': 'Villarreal',
        'visitante': 'Granada',
        'categoria_local': 'B',
        'categoria_visitante': 'C',
        'base': 0.75
    },
    {
        'id': 6,
        'local': 'Alavés',
        'visitante': 'Levante',
        'categoria_local': 'B',
        'categoria_visitante': 'B',
        'base': 0.9
    },
    {
        'id': 7,
        'local': 'Espanyol',
        'visitante': 'Sevilla',
        'categoria_local': 'B',
        'categoria_visitante': 'B',
        'base': 0.9
    },
    {
        'id': 8,
        'local': 'Betis',
        'visitante': 'Valladolid',
        'categoria_local': 'B',
        'categoria_visitante': 'C',
        'base': 0.75
    },
    {
        'id': 9,
        'local': 'Atlético',
        'visitante': 'Getafe',
        'categoria_local': 'B',
        'categoria_visitante': 'B',
        'base': 0.9
    }
]

# Cada clave es el ID del partido, y el valor es una tupla (día, hora)
asignacion_horarios = {
    0: ('Viernes', 20),
    1: ('Sábado', 12),
    2: ('Sábado', 16),
    3: ('Sábado', 18),
    4: ('Sábado', 20),
    5: ('Domingo', 16),
    6: ('Domingo', 16),
    7: ('Domingo', 18),
    8: ('Domingo', 20),
    9: ('Lunes', 20)
}

# Coeficientes de ponderación por horario
```

```
coeficientes_ponderación = {
    ('Viernes', 20): 0.40,
    ('Sábado', 12): 0.55,
    ('Sábado', 16): 0.70,
    ('Sábado', 18): 0.80,
    ('Sábado', 20): 1.00,
    ('Domingo', 12): 0.45,
    ('Domingo', 16): 0.75,
    ('Domingo', 18): 0.85,
    ('Domingo', 20): 1.00,
    ('Lunes', 20): 0.40
}

# Penalización por coincidencias
penalización = {
    0: 0.00,   # 0 coincidencias -> 0% de penalización
    1: 0.25,   # 1 coincidencia  -> 25% ...
    2: 0.45,   # 2 coincidencias -> 45%
    3: 0.60,
    4: 0.70,
    5: 0.75,
    6: 0.78,
    7: 0.80,
    8: 0.80
}
```

Cuando se fueron definiendo las funciones para calcular los algoritmos, tuvimos que hacer pruebas con los datos, nos dimos cuenta que no era lo más práctico, porque al calcular la audiencia, no es solo multiplicar la base por el coeficiente del horario sino que existen complicaciones debido a la penalización por coincidencia.

Si dos partidos coinciden en el mismo horario, su audiencia se reduce, y si tengo los partidos sueltos en una lista, para saber cuántos coinciden en cada horario tengo que recorrer todos los partidos cada vez que quiero calcular la audiencia. En general si estoy probando muchas asignaciones, eso se vuelve muy lento.

Notamos que la estructura clave no es el partido, sino el horario. Lo que realmente se necesita es agrupar los partidos por horario, de modo que cuando quiera calcular la audiencia, consulto horario por horario.

Entonces, la estructura que mejor se adapta es un diccionario que agrupa los partidos por horario, porque el cálculo de la audiencia depende directamente de cuántos partidos coinciden en el mismo horario. Su clave es el horario (por ejemplo, ('Sábado', 20:00)) y el valor es una lista de los IDs de los partidos que se juegan en ese horario. Aparte, tengo una lista de partidos donde cada partido tiene sus datos fijos: local, visitante y categoría. Cada partido tiene un ID, y en la lista del horario guardo ese ID. Así no estoy duplicando datos.

In [3]:
```
# Asignación de horario -> lista de IDs de partidos (cumple restricciones)
asignacion = {
    ('Viernes', 20): [0],
    ('Sábado', 12): [1],
    ('Sábado', 16): [2],
    ('Sábado', 18): [3],
    ('Sábado', 20): [4],
    ('Domingo', 12): [],
    ('Domingo', 16): [5, 6],
    ('Domingo', 18): [7],
    ('Domingo', 20): [8],
    ('Lunes', 20): [9]
}
```

Con esa estructura podría definir los siguientes pasos:

- Miramos los partidos que hay en ese horario.
- Para cada uno, calculamos su audiencia base por el coeficiente.
- Miramos cuántos partidos hay en total en ese horario.
- Aplicamos la penalización correspondiente a todos ellos.
- Sumamos y pasamos al siguiente horario.

Esto es mucho más eficiente, porque no tengo que estar consultando en qué horario está cada partido, sino que directamente voy a la fuente del horario que me dice qué partidos tiene. Además, si luego quiero cambiar de algoritmo (por ejemplo, pasar de fuerza bruta a genético), la estructura sigue siendo válida.

> **Sobre el modelo de estado:** Aunque el diccionario agrupado por horarios es la estructura óptima para la *evaluación* rápida de la función objetivo, la estructura de datos que mejor representa un *estado o cromosoma* en el espacio de soluciones es un **array (o lista) unidimensional de 10 elementos**. En este array, el índice representa el ID del partido y el valor almacenado es el ID del horario asignado (ej. `[0, 1, 2, 3, 4, 6, 6,`

> `7, 8, 9]`). Esta estructura lineal es fundamental porque minimiza el uso de memoria y facilita enormemente las operaciones de mutación o transición de estados en algoritmos de búsqueda local o metaheurísticas.

Según el modelo para el espacio de soluciones

(*)¿Cual es la función objetivo?

La función objetivo es la audiencia total de la jornada, que se calcula como la suma de las audiencias de todos los partidos.

Matemáticamente, se expresa como:

$$Total\ de\ audiencia = \sum_{partido=1}^{10} Audiencia(partido)$$

Donde la audiencia de cada partido se calcula como:

$$Audiencia(partido) = AudienciaBase\ x\ Ponderación\ x\ FactorPenalización$$

De esto tendriamos la ecuación en base a las relaciones de dependencia del problema:

$$Audiencia\ Total = \sum_{partido=1}^{10} AudienciaBase_{partido}\ x\ Ponderación(horario_{partido})\ x\ FactorPenalización(coincidencias($$

(*)¿Es un problema de maximización o minimización?

Además es un problema de **Maximización** para la audiencia total de la jornada, donde se requiere encontrar la asignación de partidos a horarios que hagan que el mayor número posible de personas vean los partidos.

Con esto tendríamos que encontrar el máximo valor:

$$Audiencia\ Total\ Máxima = max \sum_{partido=1}^{10} AudienciaBase_{partido}\ x\ Ponderación(horario)_{partido}\ x\ FactorPenalización(($$

Diseña un algoritmo para resolver el problema por fuerza bruta

Respuesta

Los datos de audiencia base provienen de la tabla proporcionada en el problema, que relaciona la audiencia con la combinación de categorías (A, B, C) de los equipos que se enfrentan.

Cada partido será un diccionario que contiene:

- id: identificador numérico del partido
- local: nombre del equipo local
- visitante: nombre del equipo visitante
- base: audiencia en millones de espectadores, referencia con coeficiente 1, que se generaría si se jugara el sábado a las 20:00

In [4]:
```python
# Lista de los 10 horarios disponibles. Cada horario es una tupla (día, hora).
horarios = [
    ('Viernes', 20),  # índice 0
    ('Sábado', 12),   # índice 1
    ('Sábado', 16),
    ('Sábado', 18),
    ('Sábado', 20),
    ('Domingo', 12),
    ('Domingo', 16),
    ('Domingo', 18),
    ('Domingo', 20),
    ('Lunes', 20)     # índice 9
]

# Coeficientes de ponderación por horario (clave = tupla de la lista anterior)
```

```
coeficientes = {
    ('Viernes', 20): 0.40,
    ('Sábado', 12): 0.55, ('Sábado', 16): 0.70, ('Sábado', 18): 0.80, ('Sábado', 20): 1.00,
    ('Domingo', 12): 0.45, ('Domingo', 16): 0.75, ('Domingo', 18): 0.85, ('Domingo', 20): 1.00,
    ('Lunes', 20): 0.40
}
```

In [5]:
```
# Diccionario de penalización.
penalizacion = {
    0: 0.00,
    1: 0.25,
    2: 0.45,
    3: 0.60,
    4: 0.70,
    5: 0.75,
    6: 0.78,
    7: 0.80,
    8: 0.80   # 8 coincidencias (9 partidos) → 80%
}
```

Vamos a definir una función calcular_audiencia, que reciba como entrada una lista "asignacion" de 10 elementos, donde asignacion[i] es el índice del horario asignado al partido i.

Ejemplos:

- Todos los partidos están asignados al horario con índice 0, que es Viernes 20:00

$$asignacion = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$$

- Los partidos con id 5 y 6 (Villarreal - Granada y Alavés - Levante) comparten horario (Domingo 16:00). Se infiere que hay 1 coincidencia, por ende una penalización del 25%.

$$asignacion = [0, 1, 2, 3, 4, 6, 6, 7, 8, 9]$$

- Los primeros 5 partidos están en el horario de índice 4 (Sábado 20:00), tiene 4 coincidencias que se traducen en una penalización de 70%. Similarmente los siguientes 3 partidos apuntan al índice 8 de horarios, que corresponde al Domingo a las 20:00, tiene 2 coincidencias que se traducen en una penalización de 45%. Los últimos partidos corresponden al lunes y viernes.

$$asignacion = [4, 4, 4, 4, 4, 8, 8, 8, 9, 0]$$

- El algoritmo está en medio del proceso asignando los partidos con id 0, 1 y 2, pero los partidos con id 3 al 9, todavía no tienen horario, por lo que podemos representar los demás elementos con -1

$$asignacion = [4, 7, 2, -1, -1, -1, -1, -1, -1, -1]$$

Debemos tener en cuenta, las restricciones que exista al menos un partido asignado tanto al lunes y el viernes. Por ello, debe existir en la lista asignación por lo menos un 9 y 0, sino no es válida.

El retorno de la función nos debe dar la audiencia total en millones de espectadores.

In [6]:
```
def calcular_audiencia(asignacion):
    # Agrupemos los partidos por horario en un diccionario horario_partidos
    # Ejemplo: {('Sábado', 20): [0, 4], ('Domingo', 16): [5, 6], ...}
    horario_partidos = {} # Inicializa vacío

    # Recorremos cada partido para saber en qué horario juega.
    for id_partido, id_horario in enumerate(asignacion):
        if id_horario == -1:
            continue # Ignorar partidos no asignados (vital para el Beam Search)

        horario = horarios[id_horario]
        if horario not in horario_partidos:
            horario_partidos[horario] = []
        horario_partidos[horario].append(id_partido)

    # Calculemos la audiencia por horario.
    total = 0  # Variable para acumular la audiencia total

    # Recorremos cada horario y la lista de partidos que tiene asignados.
    for horario, ids_partidos in horario_partidos.items():
        # Número de coincidencias = número de partidos en ese horario - 1.
        # Ejemplo: si hay 3 partidos → 2 coincidencias.
        num_coincidencias = len(ids_partidos) - 1

        # Factor de penalización: 1 - penalización.
```

```
        # Ejemplo: penalización 0.25 → factor 0.75.
        # Usamos .get() para evitar KeyError si hay 9 o más coincidencias
        factor_penalizacion = 1 - penalizacion.get(num_coincidencias, 0.80)

        # Coeficiente del horario
        # Ejemplo: Sábado 20:00 → 1.00
        coef_horario = coeficientes[horario]

        # Calculamos la audiencia de cada partido en este horario.
        for id_partido in ids_partidos:
            total += partidos[id_partido]['base'] * coef_horario * factor_penalizacion

    return total
```

Además de la penalización, para cada asignación, como mencionamos, deben cumplirse las siguientes restricciones obligatorias:
Al menos 1 partido debe jugarse el viernes a las 20:00.
Al menos 1 partido debe jugarse el lunes a las 20:00.

Para verificar estas restricciones, definimos una función ¨restricciones¨ que reciba de entrada a la lista de ¨asignación¨ y devuelva la veracidad lógica si se cumplen ambas restricciones (True o False)

In [7]:
```
def restricciones(asignacion):
    # Días en los que hay al menos un partido (ignoramos los no asignados: -1)
    dias = {horarios[h][0] for h in asignacion if h != -1}
    return 'Viernes' in dias and 'Lunes' in dias
```

Nuestro algoritmo de fuerza bruta, necesita encontrar la asignación que maximice la audiencia, por eso requiere generar todas las asignaciones posibles. Mientras realiza cada asignación debe pasar al proceso de cálculo de audiencia y restricción con las funciones predifinidas para comparar cuál es la mayor válida de todas.

Para generar todas las combinaciones, usaremos la técnica de backtracking para la asignación, que las genera en orden lexicográfico, que es similar a decir orden alfabético pero para los números. En nuestro caso para los 10 partidos y 10 horarios, tendríamos la siguiente lógica aplicada del backtracking:

[0,0,0,0,0,0,0,0,0,0] ← todos a 0
[0,0,0,0,0,0,0,0,0,1] ← último avanza a 1
[0,0,0,0,0,0,0,0,0,2] ← último avanza a 2
[0,0,0,0,0,0,0,0,0,3] ← último avanza a 3
[0,0,0,0,0,0,0,0,0,4] ← último avanza a 4
[0,0,0,0,0,0,0,0,0,5] ← último avanza a 5
[0,0,0,0,0,0,0,0,0,6] ← último avanza a 6
[0,0,0,0,0,0,0,0,0,7] ← último avanza a 7
[0,0,0,0,0,0,0,0,0,8] ← último avanza a 8
[0,0,0,0,0,0,0,0,0,9] ← último llega a 9

[0,0,0,0,0,0,0,0,1,0] ← penúltimo pasa a 1, último reinicia a 0
[0,0,0,0,0,0,0,0,1,1] ← último avanza a 1
[0,0,0,0,0,0,0,0,1,2] ← último avanza a 2
[0,0,0,0,0,0,0,0,1,3] ← último avanza a 3
...
[0,0,0,0,0,0,0,0,1,9] ← último llega a 9
[0,0,0,0,0,0,0,0,2,0] ← penúltimo pasa a 2, último reinicia a 0
[0,0,0,0,0,0,0,0,2,1] ← último avanza a 1
...
[0,0,0,0,0,0,0,0,9,9] ← penúltimo en 9, último en 9

[0,0,0,0,0,0,0,1,0,0] ← posición 7 pasa a 1, últimos dos reinician a 0
[0,0,0,0,0,0,0,1,0,1] ← último avanza
[0,0,0,0,0,0,0,1,0,2] ← último avanza
...
[0,0,0,0,0,0,0,1,9,9] ← últimos dos llegan a 9
[0,0,0,0,0,0,0,2,0,0] ← posición 7 pasa a 2
...

Y asi secuencialmente hasta llegar a la última combinación
[9,9,9,9,9,9,9,9,9,9]

Vamos a definir la función de ejecución principal del algoritmo de fuerza bruta "algoritmo_fuerzabruta" que permita realizar el backtracking y llevarlo a los cálculos. Esta función debe retornar principalmente:

- mejor_asignacion: lista con la asignación de horarios que maximiza la audiencia.
- mejor_audiencia: valor de la audiencia máxima alcanzada en millones de espectadores.

Adicional podemos retornar la cantidad de evaluaciones hechas.

In [8]:
```
def algoritmo_fuerzabruta():
    # Inicializamos
    mejor_asignacion = None       # No existe aun mejor asignación
    mejor_audiencia = -1          # Aseguramos comparación favorable contra cualquier audiencia resultante no neg
    evaluaciones = 0              # Contador de asignaciones evaluadas
    asignacion_actual = [0] * 10  # Todos los partidos al horario 0.

    # Creo la función interna de backtracking. La entrada es el índice del partido (0-9)
    def backtracking(índice):
        # Permite modificar variables de la función externa.
        nonlocal mejor_asignacion, mejor_audiencia, evaluaciones

        # Se termina una combinación para la lista de asignación actual cada vez que se llega a backtracking(10)
        if índice == 10:
            # Verificamos si cumple las restricciones (viernes y lunes) llamando a la función restricciones.
            if restricciones(asignacion_actual):
                # Incrementamos el contador de asignaciones evaluadas.
                evaluaciones += 1

                # Calculamos la audiencia de esta asignación llamando a la función calcular_audiencia.
                audiencia = calcular_audiencia(asignacion_actual)

                # Si es mejor que la que teníamos, la guardamos.
                if audiencia > mejor_audiencia:
                    mejor_audiencia = audiencia
                    # Para cada mejor audiencia evaluada, copiamos la lista de asignacion_actual evitando reflejo
                    mejor_asignacion = asignacion_actual.copy()

            return  # Esto generará que el backtracking(10), vuelva a su estado anterior con backtracking(9)
                    # Cada retorno a backtracking(9) tomará el id_horario + 1 de su for, hasta terminarlo, result
                    # [0,0,0,0,0,0,0,0,0,1]
                    # [0,0,0,0,0,0,0,0,0,2]
                    # ...
                    # [0,0,0,0,0,0,0,0,0,9]
                    # Acabado el for, backtracking(9) tendrá que retornar a su estado anterior con backtracking(8
                    # El backtracking(9) tendría que reiniciarse de 0 a 9, provocado por el retorno condicional d
                    # Resultando lo siguiente:
                    # [0,0,0,0,0,0,0,0,1,0]
                    # [0,0,0,0,0,0,0,0,1,1]
                    # [0,0,0,0,0,0,0,0,1,2]
                    # ...
                    # [0,0,0,0,0,0,0,0,1,9]
                    # Acabado el for, backtracking(9) tendrá que retornar a su estado anterior con backtracking(8
                    # Sucederá una misma lógica donde backtracking(10) provoque el reinicio de backtracking(9), r
                    # [0,0,0,0,0,0,0,0,2,0]
                    # [0,0,0,0,0,0,0,0,2,1]
                    # [0,0,0,0,0,0,0,0,2,2]
                    # ...
                    # [0,0,0,0,0,0,0,0,2,9]
                    # Seguirá esta secuencia hasta [0,0,0,0,0,0,0,0,9,9], donde termina el for para backtracking(
                    # Tendrá que retornar al estado anterior de backtracking(7) con un id_horario + 1
                    # Por la recursión backtracking(8) reiniciará en 0, llama a backtracking(9) y también reinici
                    # Nuevamente backtracking(10) provocará el recorrido de backtracking(9) y este al retornar au
                    # Resultando lo siguiente:
                    # [0,0,0,0,0,0,0,1,0,0]
                    # [0,0,0,0,0,0,0,1,0,1]
                    # [0,0,0,0,0,0,0,1,0,2]
                    # ...
                    # [0,0,0,0,0,0,0,1,9,9]
                    # [0,0,0,0,0,0,0,2,0,0]
                    # ...
                    # Secuencialmente hasta el último
                    # [9,9,9,9,9,9,9,9,9,9]


        # Recorremos los horarios(índices 0-9). Un range(10) directo quitaría escalabilidad, por si se modifica l
        for id_horario in range(len(horarios)):
            # Asignamos este id_horario al partido actual.
            asignacion_actual[índice] = id_horario
            # Llamada recursiva a backtracking, para pasar a la siguiente asignación.
```

```
            backtracking(índice + 1)

    #Iniciamos la función backtracking con la primera posición del primer partido (índice 0).
    backtracking(0)

    # Devolvemos la mejor solución encontrada.
    return mejor_asignacion, mejor_audiencia, evaluaciones
```

Por fuerza bruta con 10 partidos, como hemos calculado, el espacio de búsqueda es mas de 4.100 millones de combinaciones, por lo que la ejecución completa no es viable en un ordenador convencional. Pero podemos plantear el código para mostrar los resultados del algoritmo:

In [9]:
```
# Seguridad para que se ejecute solo cuando el script corra localmente.
if __name__ == "__main__":
    print("ADVERTENCIA: La ejecución completa de fuerza bruta para N=10 tomará horas/días.")
    print("Se omite la ejecución de algoritmo_fuerzabruta() en este bloque para evitar bloqueos.")
    # mejor_asignacion, mejor_audiencia, evaluaciones = algoritmo_fuerzabruta()
    # print(f"Asignaciones evaluadas: {evaluaciones}")
    # print(f"Mejor audiencia: {mejor_audiencia} millones")
```

```
ADVERTENCIA: La ejecución completa de fuerza bruta para N=10 tomará horas/días.
Se omite la ejecución de algoritmo_fuerzabruta() en este bloque para evitar bloqueos.
```

Calcula la complejidad del algoritmo por fuerza bruta

Respuesta

> Determinando el total de combinaciones sin restricciones: Sea N el número de partidos y H el número de horarios. Cada partido puede ir a cualquiera de los H horarios. Por lo tanto, el tamaño del espacio de búsqueda es H^N.
>
> Para evaluar cada combinación, el algoritmo realiza operaciones proporcionales a N (agrupar partidos, calcular penalizaciones y verificar restricciones). Esto añade un factor lineal $O(N)$.
>
> Por lo tanto, la complejidad temporal asintótica exacta del algoritmo de fuerza bruta es $O(H^N \cdot N)$. Para la instancia específica de este problema ($N = 10, H = 10$), el algoritmo realizará un número de operaciones proporcional a 10^{11}.

(*)Diseña un algoritmo que mejore la complejidad del algortimo por fuerza bruta. Argumenta porque crees que mejora el algoritmo por fuerza bruta

Respuesta

Argumentación de la mejora: El algoritmo de fuerza bruta explora a ciegas todo el espacio de búsqueda. Un Hill Climbing simple es insuficiente porque se atasca fácilmente en óptimos locales al no tener visión a futuro.

Para solucionar esto e incorporar un estimador en un árbol de expansión, implementamos un algoritmo de **Búsqueda en Haz (Beam Search)**. Este algoritmo construye la solución nivel por nivel (partido a partido). En lugar de expandir todos los nodos (como haría BFS o Fuerza Bruta), utiliza una **función heurística (estimador)** para evaluar el potencial de las asignaciones parciales. En cada nivel, solo conserva los W mejores nodos (Beam Width o anchura del haz), podando el resto del árbol.

El estimador calcula la audiencia real de los partidos ya asignados y suma una estimación optimista (audiencia máxima posible sin penalizaciones) para los partidos aún no asignados. Esto mejora radicalmente la complejidad porque poda ramas subóptimas tempranamente, pero explora con mayor profundidad y amplitud que una simple búsqueda local.

Además se incorpora una poda de factibilidad que descarta nodos que ya no pueden satisfacer las restricciones de viernes y lunes.

In [10]:
```
def estimador_heuristico(asignacion_parcial):
    # 1. Audiencia real de los partidos ya asignados
    audiencia_actual = calcular_audiencia(asignacion_parcial)
    # 2. Cota optimista de los restantes: coef 1.0 y sin coincidencias
    estimacion_futura = sum(partidos[i]['base']
                            for i, h in enumerate(asignacion_parcial) if h == -1)
    return audiencia_actual + estimacion_futura
```

```python
def algoritmo_beam_search(lista_partidos, beam_width=10):
    n = len(lista_partidos)
    estado_inicial = [-1] * n
    haz = [(estimador_heuristico(estado_inicial), estado_inicial)]

    # Expandimos primero los partidos más valiosos
    orden_partidos = sorted(range(n), key=lambda i: lista_partidos[i]['base'], reverse=True)

    for nivel in range(n):
        id_partido_actual = orden_partidos[nivel]
        partidos_restantes = n - nivel - 1
        nuevos_estados = []

        for _, estado in haz:
            for id_horario in range(len(horarios)):
                nuevo_estado = estado.copy()
                nuevo_estado[id_partido_actual] = id_horario

                # PODA DE FACTIBILIDAD: si faltan más días obligatorios (viernes/lunes)
                # que partidos por asignar, este nodo nunca llegará a una solución válida.
                dias = {horarios[h][0] for h in nuevo_estado if h != -1}
                faltan = ('Viernes' not in dias) + ('Lunes' not in dias)
                if faltan > partidos_restantes:
                    continue

                nuevos_estados.append((estimador_heuristico(nuevo_estado), nuevo_estado))

        nuevos_estados.sort(key=lambda x: x[0], reverse=True)
        haz = nuevos_estados[:beam_width]

    soluciones_validas = [(calcular_audiencia(e), e) for _, e in haz if restricciones(e)]
    if not soluciones_validas:
        return None, 0
    soluciones_validas.sort(key=lambda x: x[0], reverse=True)
    mejor_audiencia, mejor_estado = soluciones_validas[0]
    return mejor_estado, mejor_audiencia
```

(*)Calcula la complejidad del algoritmo

Respuesta

La complejidad del Beam Search depende del número de partidos (N), el número de horarios (H) y la anchura del haz (W). El árbol tiene N niveles. En cada nivel, expandimos los W estados probando los H horarios, generando $W \cdot H$ nuevos nodos. Evaluar el estimador cuesta $O(N)$. Ordenar los nodos cuesta $O((W \cdot H)\log(W \cdot H))$.

Fórmula explícita: $O(N \cdot [N \cdot W \cdot H + (W \cdot H)\log(W \cdot H)])$

Para $N = 10, H = 10, W = 10$: En el primer nivel se generan 10 nodos y en cada uno de los 9 siguientes, 100. El total es 10 + 9×100 = 910 evaluaciones como máximo (menos con la poda de factibilidad). Comparado con las 10^{10} evaluaciones de la fuerza bruta, la reducción del espacio de búsqueda es masiva. Al ser una heurística, no garantiza el óptimo global matemático, pero esta reducción radical permite encontrar soluciones de altísima calidad en milisegundos.

Comparativa de Óptimos: Fuerza Bruta vs Heurística (Problema Reducido)

Para validar la eficacia del estimador y del árbol de expansión propuesto, reducimos el problema a 6 partidos y los 10 horarios (10^6 = 1.000.000 combinaciones). Esto permite que la fuerza bruta termine en pocos segundos y podamos comparar si el Beam Search alcanza el óptimo global.

In [11]:
```python
import time
import math

# --- COMPARATIVA SÓLIDA (N=6, H=10) ---
# Usamos 6 partidos y los 10 horarios. Espacio de búsqueda: 10^6 = 1.000.000 combinaciones.
partidos_reducidos = partidos[:6]

partidos_originales = partidos
partidos = partidos_reducidos

# 1. Fuerza Bruta (Adaptada a N=6)
def bf_reducida():
    mejor_asig = None
```

```
    mejor_aud = -1
    asig = [-1]*6
    def backtrack(idx):
        nonlocal mejor_asig, mejor_aud
        if idx == 6:
            if restricciones(asig):
                aud = calcular_audiencia(asig)
                if aud > mejor_aud:
                    mejor_aud = aud
                    mejor_asig = asig.copy()
            return
        for h in range(len(horarios)):
            asig[idx] = h
            backtrack(idx + 1)
    backtrack(0)
    return mejor_asig, mejor_aud

inicio_bf = time.time()
optimo_bf_asig, optimo_bf_aud = bf_reducida()
tiempo_bf = time.time() - inicio_bf

# 2. Heurística (Beam Search)
inicio_bs = time.time()
optimo_bs_asig, optimo_bs_aud = algoritmo_beam_search(partidos, beam_width=10)
tiempo_bs = time.time() - inicio_bs

print("--- COMPARATIVA EN PROBLEMA REDUCIDO (N=6, H=10) ---")
print(f"Fuerza Bruta -> Óptimo: {optimo_bf_aud:.4f} Millones | Tiempo: {tiempo_bf:.4f} seg")
print(f"Beam Search  -> Óptimo: {optimo_bs_aud:.4f} Millones | Tiempo: {tiempo_bs:.4f} seg")

# Corrección: Comparación segura de floats
if math.isclose(optimo_bf_aud, optimo_bs_aud, rel_tol=1e-9):
    print("Resultado: La heurística HA ENCONTRADO el óptimo global matemático.")
else:
    print("Resultado: La heurística se ha quedado en un óptimo local.")

partidos = partidos_originales
```

```
--- COMPARATIVA EN PROBLEMA REDUCIDO (N=6, H=10) ---
Fuerza Bruta -> Óptimo: 4.6810 Millones | Tiempo: 1.0187 seg
Beam Search  -> Óptimo: 4.6810 Millones | Tiempo: 0.0013 seg
Resultado: La heurística HA ENCONTRADO el óptimo global matemático.
```

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

Respuesta

In [12]:

```
import random

audiencias_cat = {
    ('A', 'A'): 2.0, ('A', 'B'): 1.3, ('A', 'C'): 1.0,
    ('B', 'A'): 1.3, ('B', 'B'): 0.9, ('B', 'C'): 0.75,
    ('C', 'A'): 1.0, ('C', 'B'): 0.75, ('C', 'C'): 0.47
}

def generar_jornada_aleatoria(semilla):
    random.seed(semilla)
    categorias_disponibles = ['A']*3 + ['B']*11 + ['C']*6
    random.shuffle(categorias_disponibles)

    jornada = []
    for id_partido in range(10):
        cat_local = categorias_disponibles.pop()
        cat_visitante = categorias_disponibles.pop()

        tupla_cat = (cat_local, cat_visitante)
        if tupla_cat not in audiencias_cat:
            tupla_cat = (cat_visitante, cat_local)

        jornada.append({
            'id': id_partido,
            'local': f'Eq_{id_partido}_L',
            'visitante': f'Eq_{id_partido}_V',
            'categorias': f'{cat_local}-{cat_visitante}',
            'base': audiencias_cat[tupla_cat]
        })
    return jornada
```

```
# Generamos 3 juegos de datos diferentes para probar la robustez
juegos_de_datos = [generar_jornada_aleatoria(seed) for seed in [42, 100, 999]]
print("Se han generado 3 juegos de datos aleatorios.")
```

```
Se han generado 3 juegos de datos aleatorios.
```

Aplica el algoritmo al juego de datos generado

Respuesta

In [13]:
```
for i, jornada in enumerate(juegos_de_datos):
    partidos = jornada
    asignacion_optima, audiencia_max = algoritmo_beam_search(partidos, beam_width=15)

    print(f"\n--- RESULTADOS BEAM SEARCH - JUEGO DE DATOS {i+1} ---")
    if asignacion_optima:
        print(f"Mejor audiencia encontrada: {audiencia_max:.4f} Millones")
        for id_partido, id_horario in enumerate(asignacion_optima):
            p = partidos[id_partido]
            dia, hora = horarios[id_horario]
            print(f"  [{p['categorias']}] {p['local']} vs {p['visitante']} -> {dia} {hora}:00 (base {p['base']})")
    else:
        print("No se encontró solución válida.")
```

```
--- RESULTADOS BEAM SEARCH - JUEGO DE DATOS 1 ---
Mejor audiencia encontrada: 6.7130 Millones
  [B-A] Eq_0_L vs Eq_0_V -> Sábado 20:00 (base 1.3)
  [B-B] Eq_1_L vs Eq_1_V -> Sábado 18:00 (base 0.9)
  [C-A] Eq_2_L vs Eq_2_V -> Domingo 18:00 (base 1.0)
  [B-A] Eq_3_L vs Eq_3_V -> Domingo 20:00 (base 1.3)
  [B-C] Eq_4_L vs Eq_4_V -> Sábado 12:00 (base 0.75)
  [B-B] Eq_5_L vs Eq_5_V -> Domingo 16:00 (base 0.9)
  [C-C] Eq_6_L vs Eq_6_V -> Lunes 20:00 (base 0.47)
  [B-B] Eq_7_L vs Eq_7_V -> Sábado 16:00 (base 0.9)
  [B-C] Eq_8_L vs Eq_8_V -> Domingo 12:00 (base 0.75)
  [B-C] Eq_9_L vs Eq_9_V -> Viernes 20:00 (base 0.75)

--- RESULTADOS BEAM SEARCH - JUEGO DE DATOS 2 ---
Mejor audiencia encontrada: 6.7200 Millones
  [B-C] Eq_0_L vs Eq_0_V -> Sábado 16:00 (base 0.75)
  [C-B] Eq_1_L vs Eq_1_V -> Sábado 12:00 (base 0.75)
  [B-B] Eq_2_L vs Eq_2_V -> Domingo 16:00 (base 0.9)
  [C-B] Eq_3_L vs Eq_3_V -> Viernes 20:00 (base 0.75)
  [B-A] Eq_4_L vs Eq_4_V -> Sábado 20:00 (base 1.3)
  [C-B] Eq_5_L vs Eq_5_V -> Domingo 12:00 (base 0.75)
  [B-B] Eq_6_L vs Eq_6_V -> Sábado 18:00 (base 0.9)
  [B-A] Eq_7_L vs Eq_7_V -> Domingo 20:00 (base 1.3)
  [A-C] Eq_8_L vs Eq_8_V -> Domingo 18:00 (base 1.0)
  [C-B] Eq_9_L vs Eq_9_V -> Lunes 20:00 (base 0.75)

--- RESULTADOS BEAM SEARCH - JUEGO DE DATOS 3 ---
Mejor audiencia encontrada: 6.6835 Millones
  [A-C] Eq_0_L vs Eq_0_V -> Domingo 18:00 (base 1.0)
  [C-C] Eq_1_L vs Eq_1_V -> Viernes 20:00 (base 0.47)
  [C-C] Eq_2_L vs Eq_2_V -> Lunes 20:00 (base 0.47)
  [B-B] Eq_3_L vs Eq_3_V -> Domingo 16:00 (base 0.9)
  [B-B] Eq_4_L vs Eq_4_V -> Sábado 18:00 (base 0.9)
  [A-B] Eq_5_L vs Eq_5_V -> Sábado 20:00 (base 1.3)
  [C-B] Eq_6_L vs Eq_6_V -> Domingo 12:00 (base 0.75)
  [B-A] Eq_7_L vs Eq_7_V -> Domingo 20:00 (base 1.3)
  [B-B] Eq_8_L vs Eq_8_V -> Sábado 16:00 (base 0.9)
  [B-B] Eq_9_L vs Eq_9_V -> Sábado 12:00 (base 0.9)
```

In [14]:
```
print("--- VALIDACIÓN BF vs BEAM (N=6, H=10), 3 juegos aleatorios ---")
for semilla in [42, 100, 999]:
    partidos = generar_jornada_aleatoria(semilla)[:6]
    t0 = time.time(); a_bf, v_bf = bf_reducida(); t_bf = time.time() - t0
    t0 = time.time(); a_bs, v_bs = algoritmo_beam_search(partidos, beam_width=10); t_bs = time.time() - t0
    print(f"semilla={semilla:<4} BF={v_bf:.4f} Beam={v_bs:.4f} gap={v_bf - v_bs:.4f} "
          f"t_BF={t_bf:.1f}s t_Beam={t_bs*1000:.1f}ms óptimo={math.isclose(v_bf, v_bs, rel_tol=1e-9)}")
```

```
--- VALIDACIÓN BF vs BEAM (N=6, H=10), 3 juegos aleatorios ---
semilla=42   BF=4.8300 Beam=4.8300 gap=0.0000 t_BF=1.0s t_Beam=1.3ms óptimo=True
semilla=100  BF=4.0375 Beam=4.0375 gap=0.0000 t_BF=1.0s t_Beam=1.3ms óptimo=True
semilla=999  BF=4.1610 Beam=4.1610 gap=0.0000 t_BF=1.0s t_Beam=1.2ms óptimo=True
```

Conclusión: en N=6, tanto en la jornada real como en los 3 juegos aleatorios, el Beam Search alcanzó el óptimo (gap 0) en ~1 ms frente a ~1 s de la fuerza bruta. En N=10 la fuerza bruta es inviable (≈4.100 millones de asignaciones válidas), por lo que el

resultado es el mejor encontrado por la heurística, respaldado por la validación en tamaños reducidos.

Enumera las referencias que has utilizado(si ha sido necesario) para llevar a cabo el trabajo

Respuesta

- Brassard, G., y Bratley, P. (1997). *Fundamentos de algoritmia*.
- Documentación oficial de Python (estructuras de datos y librerías estándar): https://docs.python.org/3/
- Conceptos teóricos de heurísticas basadas en árboles de expansión, estimadores y Búsqueda en Haz (Beam Search) extraídos del material de la asignatura "Algoritmos de Optimización" (03MIAR).

Describe brevemente las lineas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

Respuesta

1. **Resolución Exacta (Programación Lineal Entera):** Dado que el tamaño actual del problema es reducido ($N = 10$), se podría modelar matemáticamente y resolver utilizando *solvers* como PuLP o Gurobi. Esto garantizaría encontrar el óptimo global matemático en fracciones de segundo, eliminando la incertidumbre de las heurísticas.
2. **Escalabilidad a Temporadas Completas:** Si el problema escala de 1 jornada (10 partidos) a una temporada completa (380 partidos), el árbol de expansión actual crecería demasiado. Sería necesario implementar metaheurísticas poblacionales como Algoritmos Genéticos (GA) o mejorar el estimador actual utilizando técnicas de *Machine Learning* para predecir la viabilidad de una rama antes de expandirla.
3. **Restricciones Temporales Dinámicas:** En un escenario real, La Liga impone restricciones de descanso. Por ejemplo, un equipo que juega el lunes no puede jugar el viernes de la siguiente jornada, o equipos con competiciones europeas tienen días bloqueados. Esto transformaría el problema en un Problema de Satisfacción de Restricciones (CSP) con ventanas de tiempo, requiriendo matrices de adyacencia temporal.